CEFET-MG | DEPARTAMENTO DE COMPUTAÇÃO

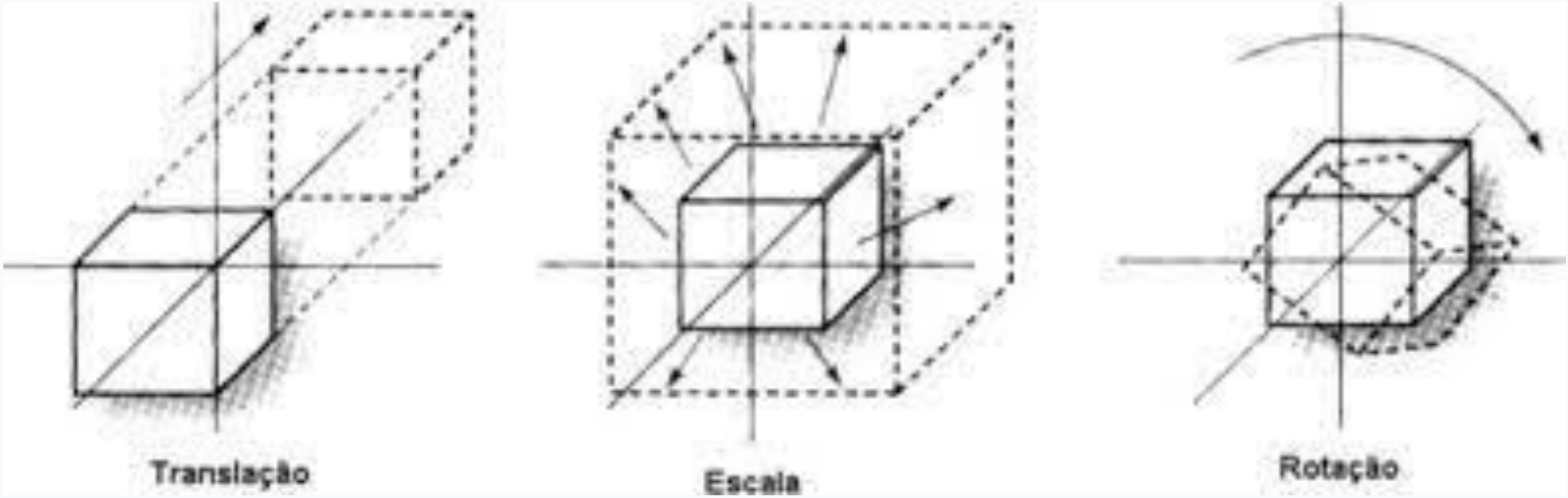
Computação Gráfica

Transformações Gráficas: Transformações Geométricas

Prof. Dr. Raulivan Rodrigo

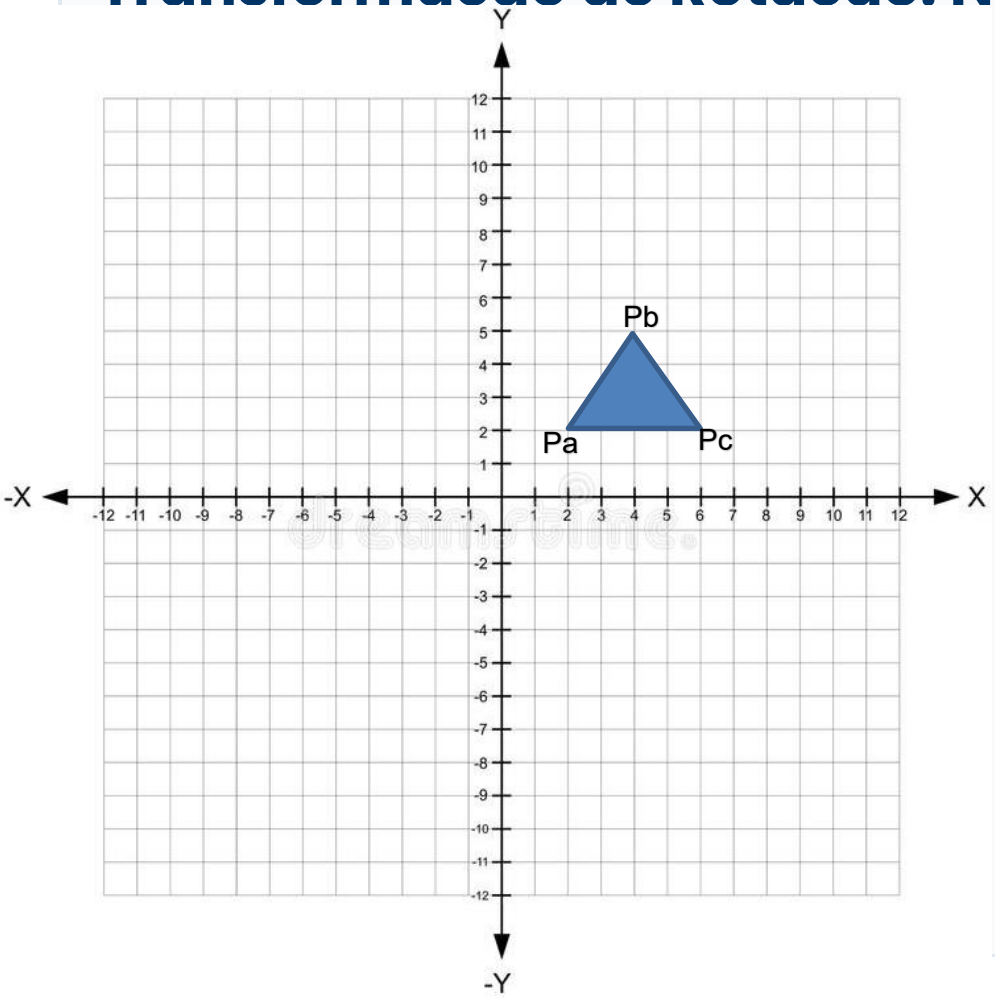
Contextualização

Em síntese as transformações geométricas são mudanças que alteram a posição, a direção ou o tamanho de um objeto no plano, criando um novo objeto a partir da original



Transformações Gráficas: Transformações Geométricas

Transformacão de Rotacão: Na prática



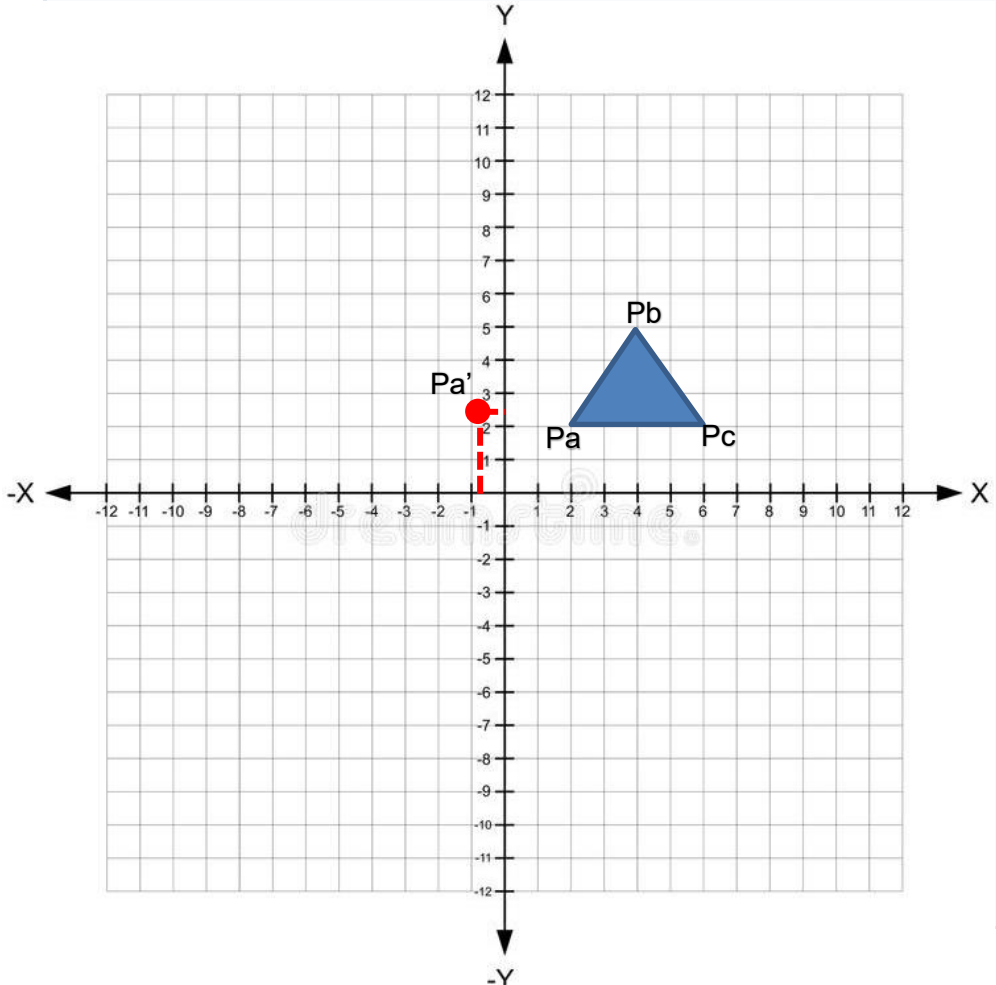
Vamos aplicar a rotação de 60º em uma imagem 2D

Pa = (2,2);
Pb = (4,5);
Pc = (6,2);

	sen	cos	tg
30°	$\frac{1}{2}$	$\frac{\sqrt{3}}{2}$	$\frac{\sqrt{3}}{3}$
45°	$\frac{\sqrt{2}}{2}$	$\frac{\sqrt{2}}{2}$	1
60°	$\frac{\sqrt{3}}{2}$	$\frac{1}{2}$	$\sqrt{3}$

Transformações Gráficas: Transformações Geométricas

Transformação de Rotação: Na prática



Rotação de 60º - Calculo do ponto Pa' de Pa(2,2)

$$\text{Cosseno}\,(a) = \frac{\text{cateto adjacente}}{\text{hipotenusa}}$$

$$\text{Seno}\,(a) = \frac{\text{cateto oposto}}{\text{hipotenusa}}$$

x2 = x1 cos(θ) – y1 sen(θ)

x2 = 2 cos(60) – 2 sen(60)

$x2 = 2\,\frac{1}{2} - 2\,\frac{\sqrt{3}}{2}$

x2 = 2*0,5 – 2 * 0,866

x2 = 1 – 1,732

X2 = - 0,732

y2 = y1 cos(θ) + x1 sen(θ)

y2 = 2 cos(60) + 2 sen(60)

$y2 = 2\,\frac{1}{2} + 2\,\frac{\sqrt{3}}{2}$

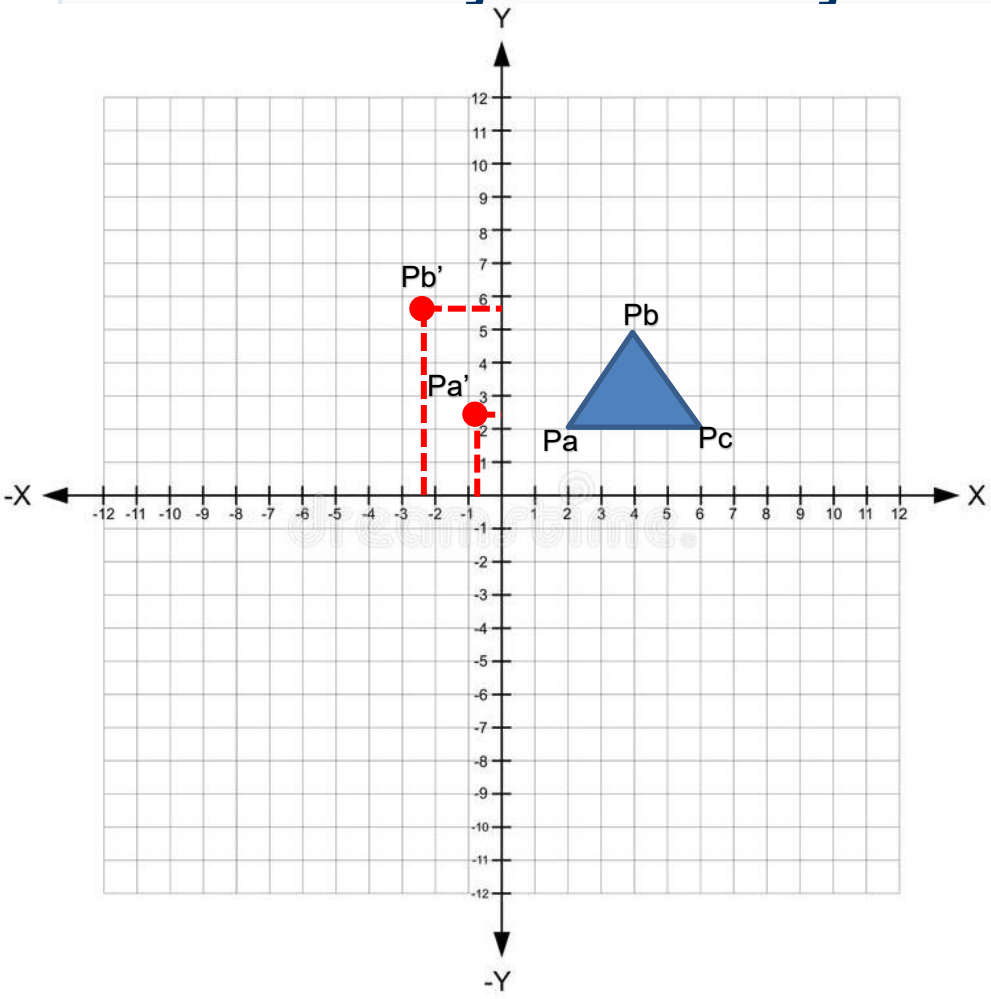
y2 = 2*0,5 + 2 * 0,866

y2 = 1 + 1,732

y2 = 2,732

Transformações Gráficas: Transformações Geométricas

Transformação de Rotação: Na prática



Rotação de 60º - Calculo do ponto Pb' de Pb(4,5)

$$\text{Cosseno}\,(a) = \frac{\text{cateto adjacente}}{\text{hipotenusa}}$$

$$\text{Seno}\,(a) = \frac{\text{cateto oposto}}{\text{hipotenusa}}$$

x2 = x1 cos(θ) – y1 sen(θ)

x2 = 4 cos(60) – 5 sen(60)

$x2 = 4\,\frac{1}{2} - 5\,\frac{\sqrt{3}}{2}$

x2 = 4*0,5 – 5 * 0,866

x2 = 2 – 4,33

X2 = - 2,33

y2 = y1 cos(θ) + x1 sen(θ)

y2 = 5 cos(60) + 4 sen(60)

$y2 = 5\,\frac{1}{2} + 4\,\frac{\sqrt{3}}{2}$

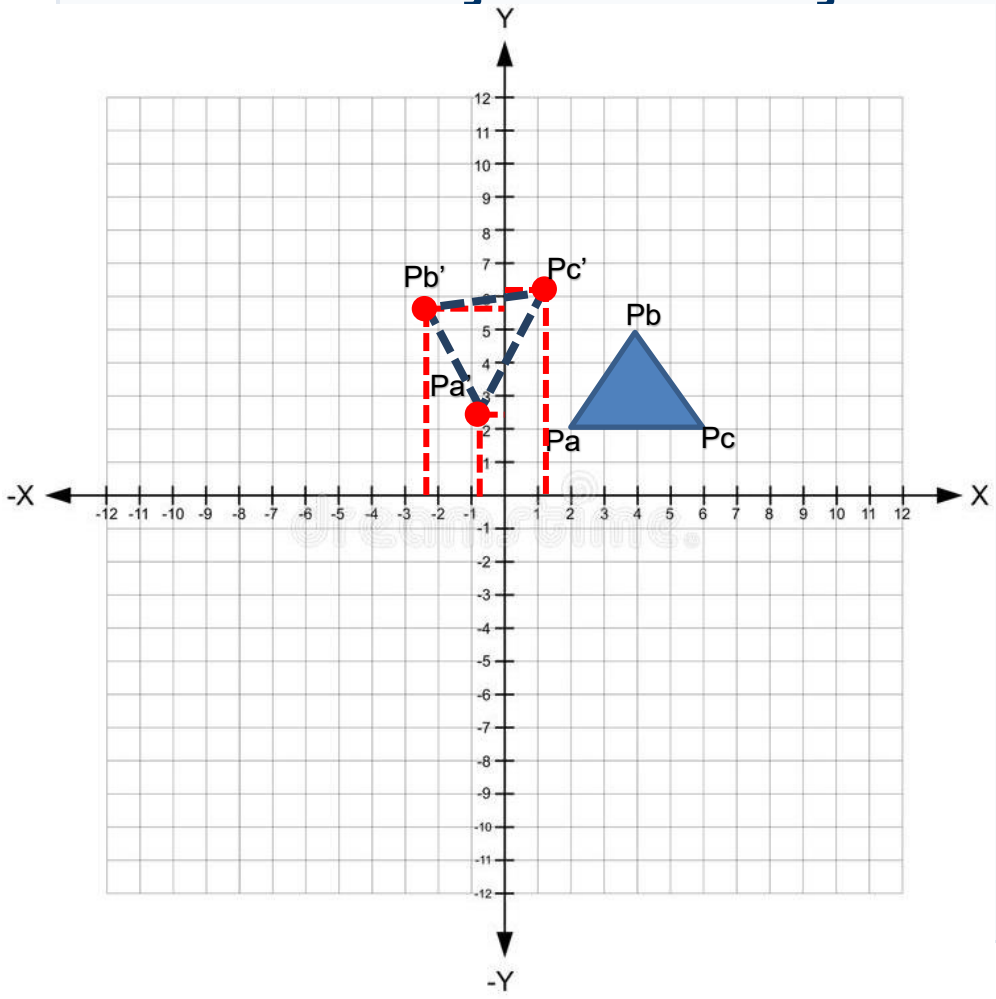
y2 = 5*0,5 + 4 * 0,866

y2 = 2,5 + 3,464

y2 = 5,964

Transformações Gráficas: Transformações Geométricas

Transformação de Rotação: Na prática



Rotação de 60º - Calculo do ponto Pc′ de Pb(6,2)

$$\text{Cosseno}\,(a) = \frac{\text{cateto adjacente}}{\text{hipotenusa}}$$

$$\text{Seno}\,(a) = \frac{\text{cateto oposto}}{\text{hipotenusa}}$$

x2 = x1 cos(θ) – y1 sen(θ)

x2 = 6 cos(60) – 2 sen(60)

$x2 = 6\,\frac{1}{2} - 2\,\frac{\sqrt{3}}{2}$

x2 = 6*0,5 – 2 * 0,866

x2 = 3 – 1,732

X2 = 1,268

y2 = y1 cos(θ) + x1 sen(θ)

y2 = 2 cos(60) + 6 sen(60)

$y2 = 2\,\frac{1}{2} + 6\,\frac{\sqrt{3}}{2}$

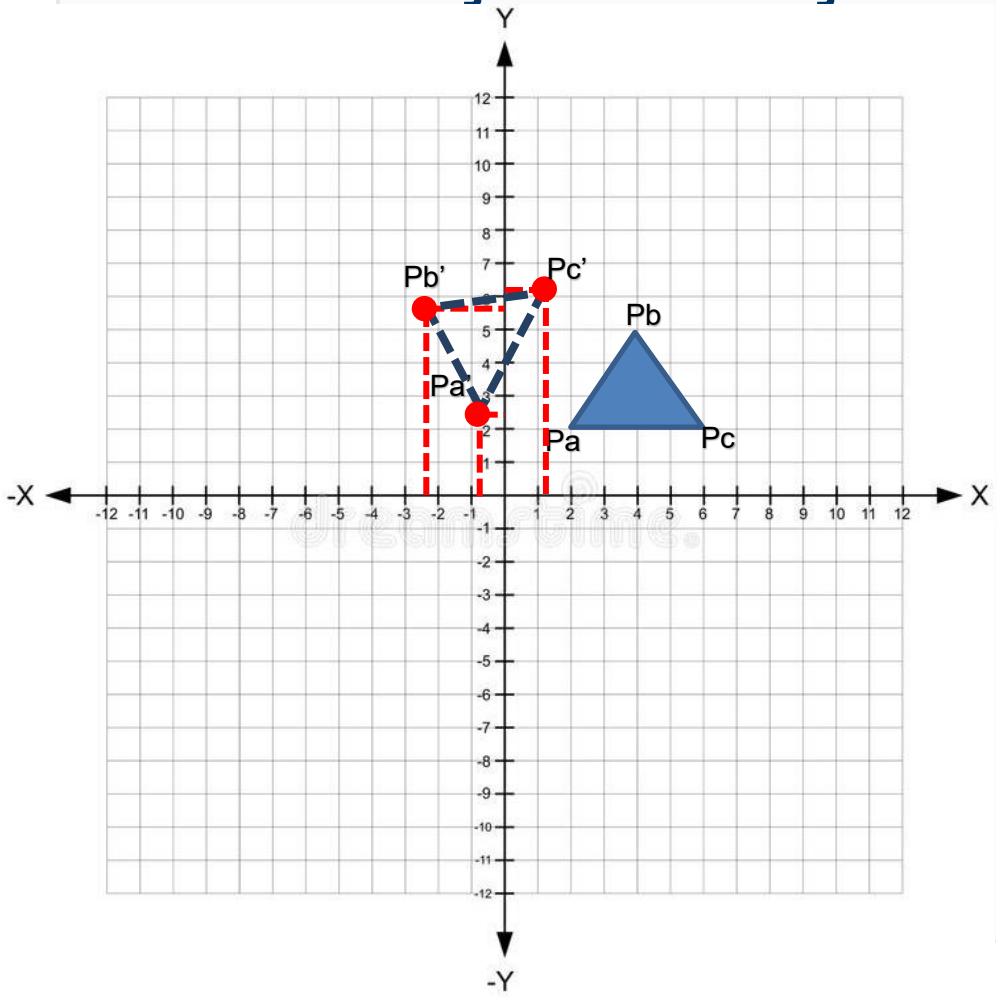
y2 = 2*0,5 + 6* 0,866

y2 = 1 + 5,196

y2 = 6,196

Transformações Gráficas: Transformações Geométricas

Transformação de Rotação: Na prática



Rotação de 60º - Calculo do ponto Pc' de Pb(6,2)

Rotação θ = 60º

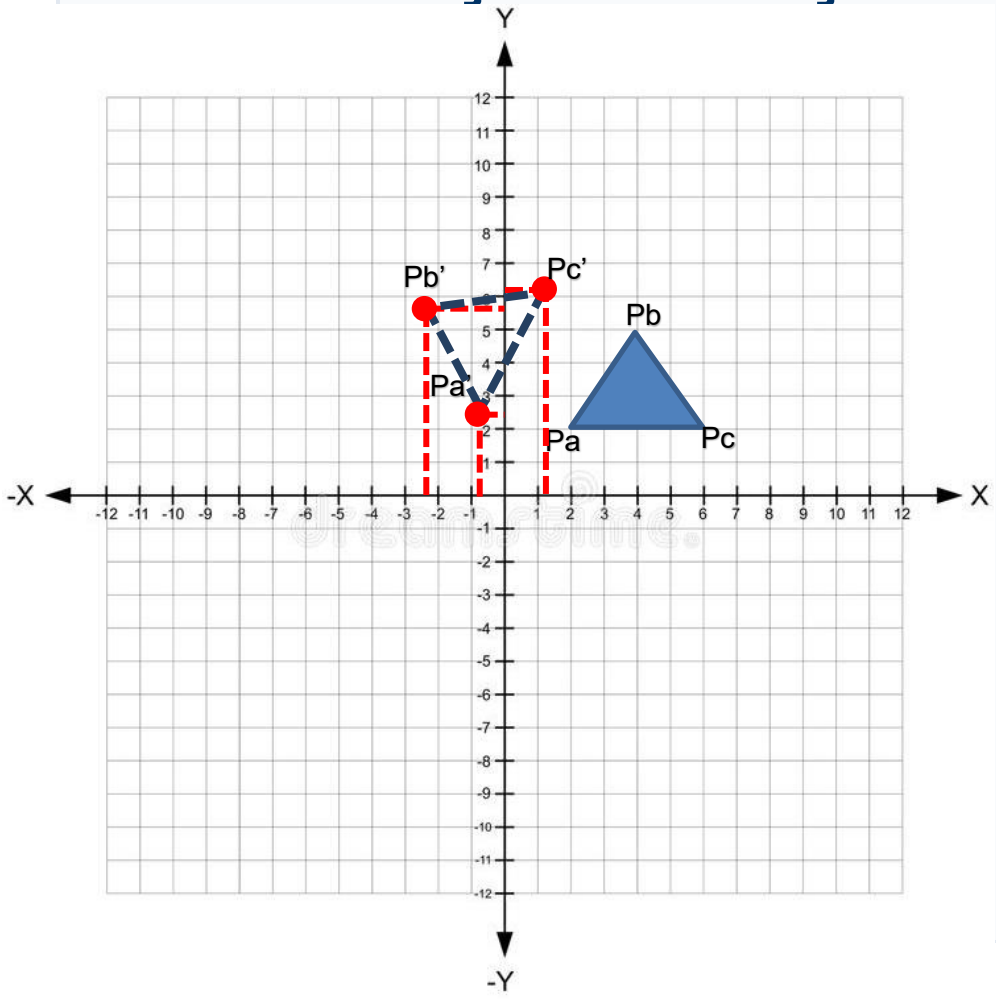
$$\begin{pmatrix} \cos(\theta) & \text{sen}(\theta) \\ -\text{sen}(\theta) & \cos(\theta) \end{pmatrix} \quad \begin{pmatrix} \frac{1}{2} & \frac{\sqrt{3}}{2} \\ -\frac{\sqrt{3}}{2} & \frac{1}{2} \end{pmatrix}$$

Pa = [2 ,2]
Pb = [4, 5] **logo**
Pc = [6, 2]

Pa' = Pa * S
Pb' = Pb * S
Pc' = Pc * S

Transformações Gráficas: Transformações Geométricas

Transformação de Rotação: Na prática



Rotação θ = 60º

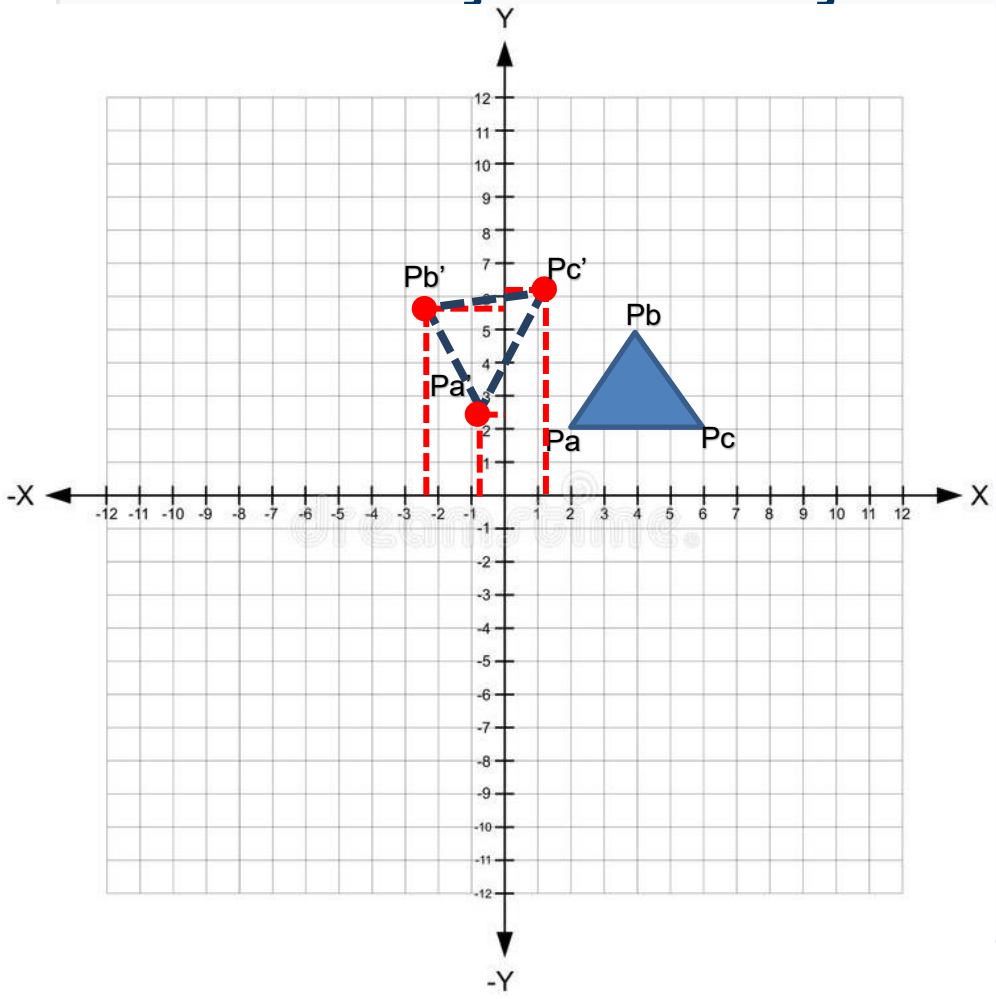
$$Pa' = Pa * S \quad Pb' = Pb * S \quad Pc' = Pc * S \quad \begin{pmatrix} \frac{1}{2} & \frac{\sqrt{3}}{2} \\ -\frac{\sqrt{3}}{2} & \frac{1}{2} \end{pmatrix}$$

$$Pa' = Pa * S \quad Pb' = Pb * S \quad Pc' = Pc * S$$

$$Pa' = [2\ ,2\ ] * \begin{pmatrix} \frac{1}{2} & \frac{\sqrt{3}}{2} \\ -\frac{\sqrt{3}}{2} & \frac{1}{2} \end{pmatrix} = [-0{,}732\ ,\ 2{,}732\ ]$$

Transformações Gráficas: Transformações Geométricas

Transformação de Rotação: Na prática



Rotação θ = 60º

$$Pa' = Pa * S$$
$$Pb' = Pb * S$$
$$Pc' = Pc * S$$

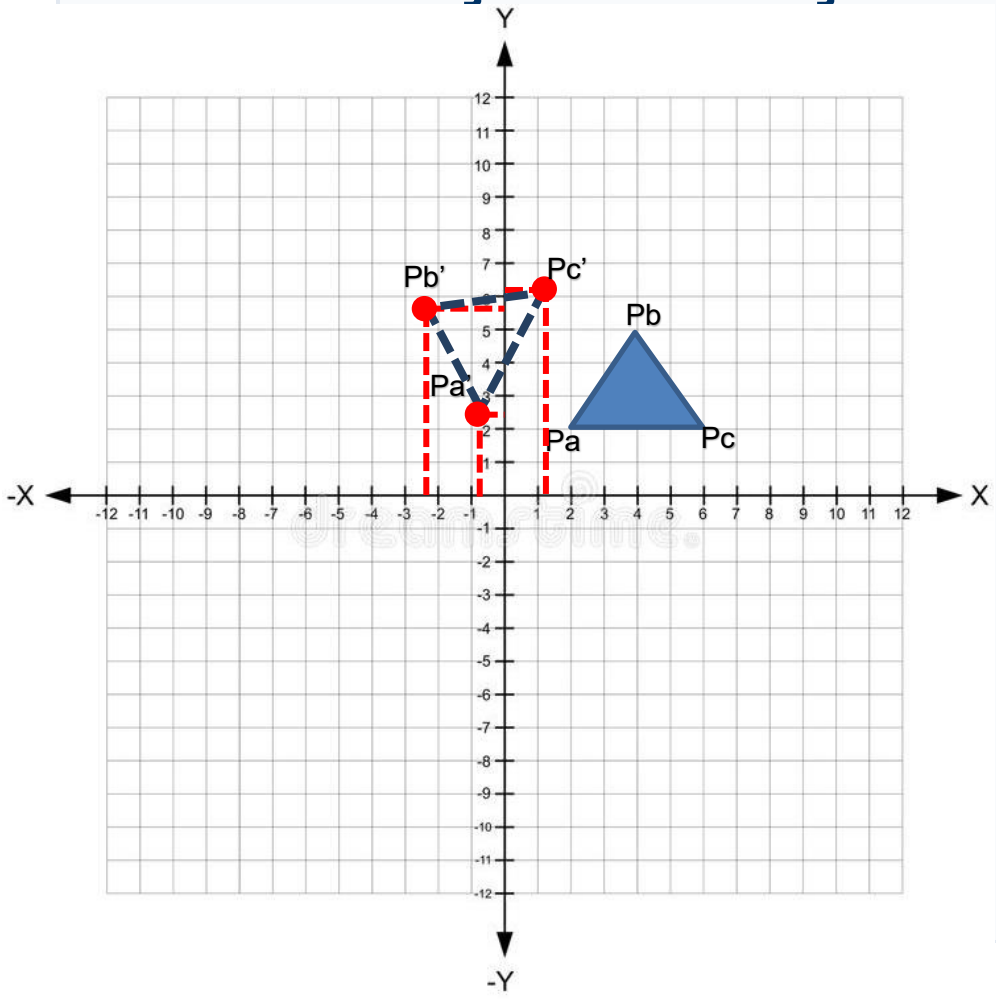
$$\begin{pmatrix} \frac{1}{2} & \frac{\sqrt{3}}{2} \\ -\frac{\sqrt{3}}{2} & \frac{1}{2} \end{pmatrix}$$

$$Pa' = Pa * S$$
$$Pb' = Pb * S$$
$$Pc' = Pc * S$$

$$Pb' = [4\ ,\ 5\ ] * \begin{pmatrix} \frac{1}{2} & \frac{\sqrt{3}}{2} \\ -\frac{\sqrt{3}}{2} & \frac{1}{2} \end{pmatrix} = [-2,\ 33\ ,\ \ 5{,}964\ ]$$

Transformações Gráficas: Transformações Geométricas

Transformação de Rotação: Na prática



Rotação θ = 60º

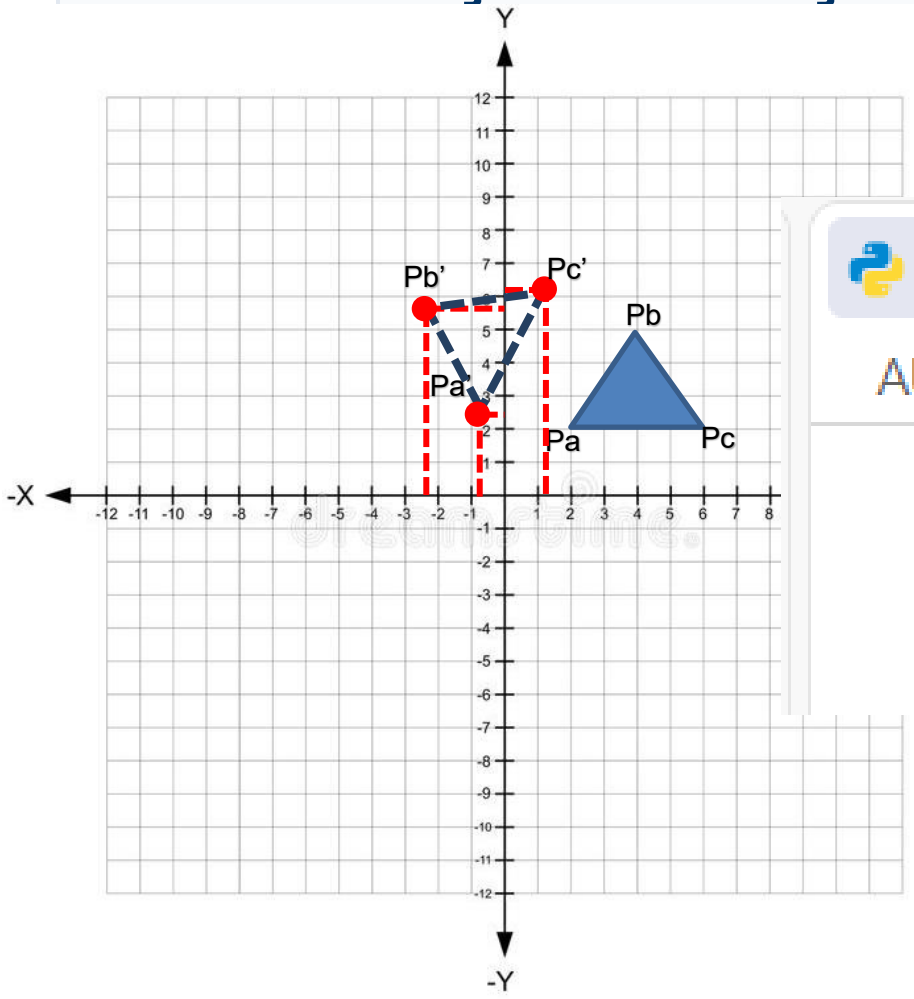
$$Pa' = Pa * S$$
$$Pb' = Pb * S$$
$$Pc' = Pc * S$$
$$\begin{pmatrix} \frac{1}{2} & \frac{\sqrt{3}}{2} \\ -\frac{\sqrt{3}}{2} & \frac{1}{2} \end{pmatrix}$$

$$Pa' = Pa * S$$
$$Pb' = Pb * S$$
$$Pc' = Pc * S$$

$$Pc' = [6\ ,\ 2\ ] * \begin{pmatrix} \frac{1}{2} & \frac{\sqrt{3}}{2} \\ -\frac{\sqrt{3}}{2} & \frac{1}{2} \end{pmatrix} = [1{,}268\ ,\ 6{,}196\ ]$$

Transformações Gráficas: Transformações Geométricas

Transformação de Rotação: Na prática



transformacao_rotacao.py

transformacao_rotacao.py U

AULA11 > transformacao_rotacao.py > transformacao_rotacao

```
1   import numpy as np
2   import matplotlib.pyplot as plt
3   from matplotlib.patches import Arc
4
```

Transformações Gráficas: Transformações Geométricas

```
5    def transformacao_rotacao():
6        print("=== Rotação 2D ===\n")
7
8        # Definição od triângulo
9        # Vértices: A(2,2), B(4,5), C(6,2) e fechamento em A(2,2)
10       matriz_vertices = np.array([
11           [2, 4, 6, 2], # Eixo X
12           [2, 5, 2, 2], # Eixo Y
13           [1, 1, 1, 1]  # Coordenada Homogênea (W)
14       ], dtype=np.float32)
15
16       print("Matriz de Vértices Original:")
17       print(matriz_vertices, "\n")
```

-X

-Y

Transformações Gráficas: Transformações Geométricas

```
    # Definição do Ângulo e Conversão para Radianos
    angulo_graus = 60
    theta = np.radians(angulo_graus)

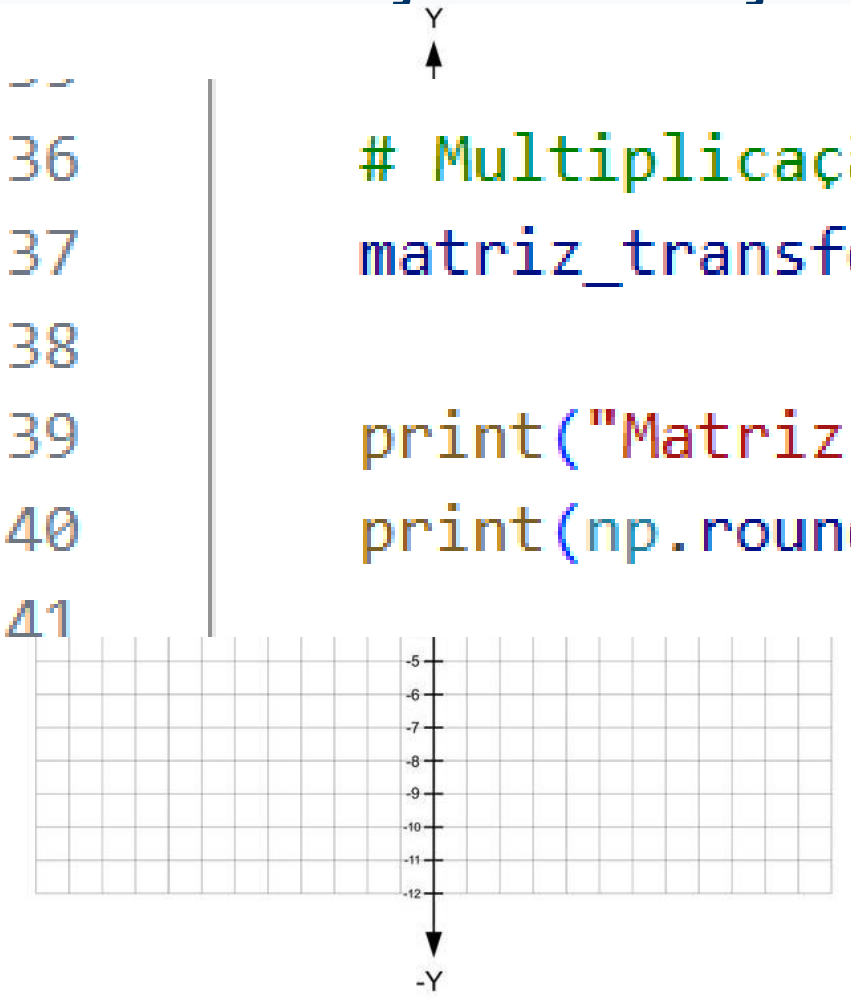
    # Construção da Matriz de Rotação
    cos_t = np.cos(theta)
    sin_t = np.sin(theta)

    matriz_rotacao = np.array([
        [ cos_t, -sin_t, 0],
        [ sin_t,  cos_t, 0],
        [     0,      0, 1]
    ])

    print(f"Matriz de Rotação ({angulo_graus} graus / {theta:.4f} radianos):")
    print(np.round(matriz_rotacao, 4), "\n")
```

Transformações Gráficas: Transformações Geométricas

Transformação de Rotação: Na prática



transformacao_rotacao.py

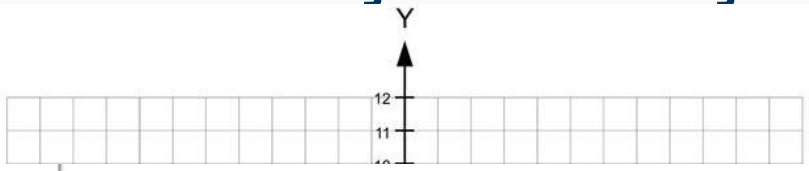
```
36      # Multiplicação
37      matriz_transformada = matriz_rotacao @ matriz_vertices
38
39      print("Matriz de Vértices Rotacionada:")
40      print(np.round(matriz_transformada, 4), "\n")
41
```

Transformações Gráficas: Transformações Geométricas

```
42      # Renderização no Matplotlib
43      fig, ax = plt.subplots(figsize=(10, 10))
44
45      x_orig, y_orig = matriz_vertices[0, :], matriz_vertices[1, :]
46      x_trans, y_trans = matriz_transformada[0, :], matriz_transformada[1, :]
47
48      # Plotando os triângulos
49      ax.plot(x_orig, y_orig, color='blue', linestyle='--',
50              linewidth=2, marker='o', label='Original')
51      ax.fill(x_orig, y_orig, color='blue', alpha=0.1)
52
53      ax.plot(x_trans, y_trans, color='green', linestyle='-',
54              linewidth=2, marker='s', label=f'Rotacionado ({angulo_graus}º)')
55      ax.fill(x_trans, y_trans, color='green', alpha=0.3)
56
```

Transformações Gráficas: Transformações Geométricas

Transformação de Rotação: Na prática



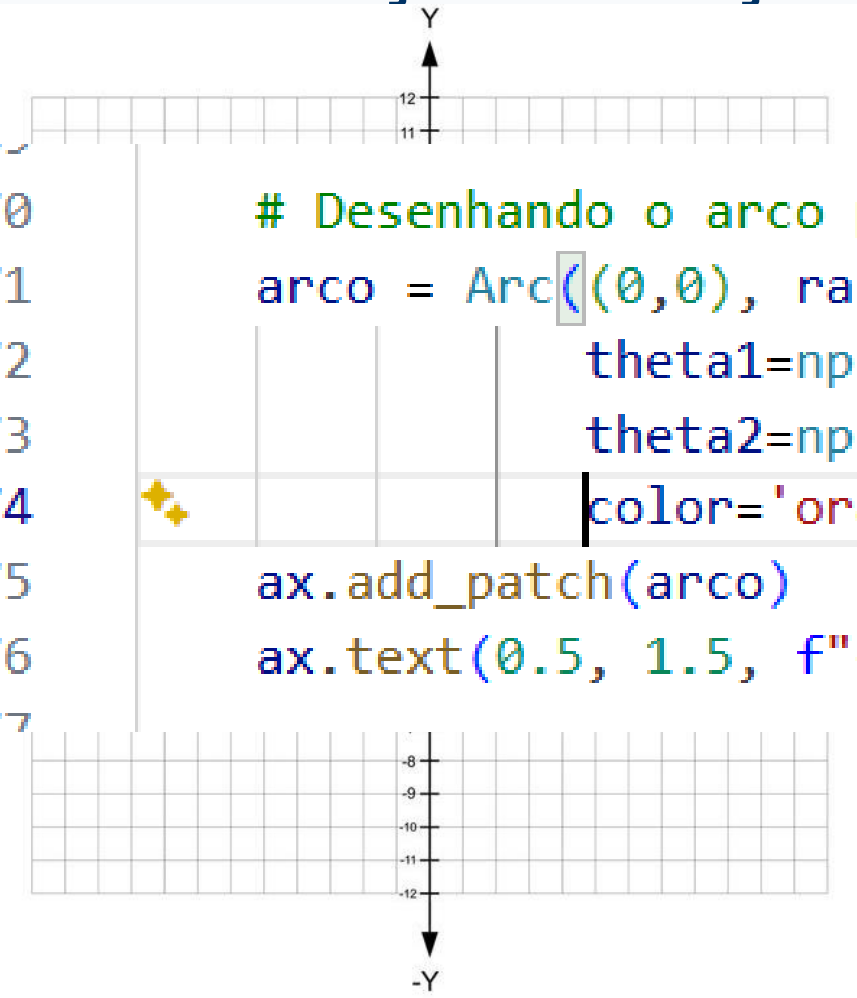
transformacao_rotacao.py

```
57      # Legendas
58      rotulos = ['A', 'B', 'C']
59      for i in range(3):
60          ax.annotate(f"{rotulos[i]}", (x_orig[i], y_orig[i]),
61                      textcoords="offset points", xytext=(5,5), color='blue')
62          ax.annotate(f"{rotulos[i]}'", (x_trans[i], y_trans[i]),
63                      textcoords="offset points", xytext=(5,5), color='darkgreen')
64
65      # Desenhando as linhas de raio e o arco de órbita do Vértice A
66      raio_a = np.sqrt(x_orig[0]**2 + y_orig[0]**2) # Distância da origem usando TeoremaPitágoras
67      ax.plot([0, x_orig[0]], [0, y_orig[0]], color='gray', linestyle=':', alpha=0.7)
68      ax.plot([0, x_trans[0]], [0, y_trans[0]], color='gray', linestyle=':', alpha=0.7)
69
```

-Y

Transformações Gráficas: Transformações Geométricas

Transformação de Rotação: Na prática



transformacao_rotacao.py

```
70      # Desenhando o arco para mostrar o movimento angular (A até A')
71      arco = Arc((0,0), raio_a*2, raio_a*2, angle=0,
72                 theta1=np.degrees(np.arctan2(y_orig[0], x_orig[0])),
73                 theta2=np.degrees(np.arctan2(y_trans[0], x_trans[0])),
74                 color='orange', linewidth=2, linestyle='--')
75      ax.add_patch(arco)
76      ax.text(0.5, 1.5, f"{angulo_graus}º", color='orange', fontweight='bold')
```

```python
    # Configuração do Plano Cartesiano
    ax.set_title("Rotação 2D Orbital", fontsize=14, fontweight='bold')

    # Ajustando limites para mostrar a rotação (quadrante negativo de X)
    ax.set_xlim(-6, 8)
    ax.set_ylim(-2, 10)

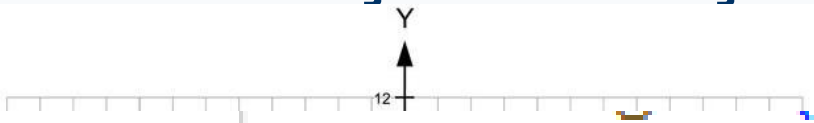
    ax.spines['left'].set_position('zero')
    ax.spines['bottom'].set_position('zero')
    ax.spines['right'].set_color('none')
    ax.spines['top'].set_color('none')

    ax.set_xticks(np.arange(-6, 9, 1))
    ax.set_yticks(np.arange(-2, 11, 1))
    ax.grid(True, linestyle=':', color='gray', alpha=0.5)

    ax.legend(loc='upper right', fontsize=12)
```

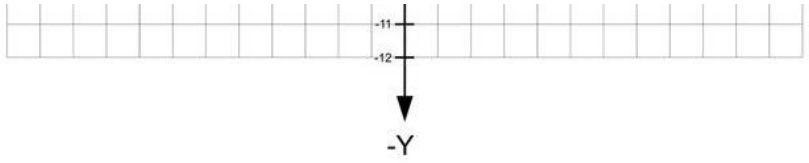
Transformações Gráficas: Transformações Geométricas

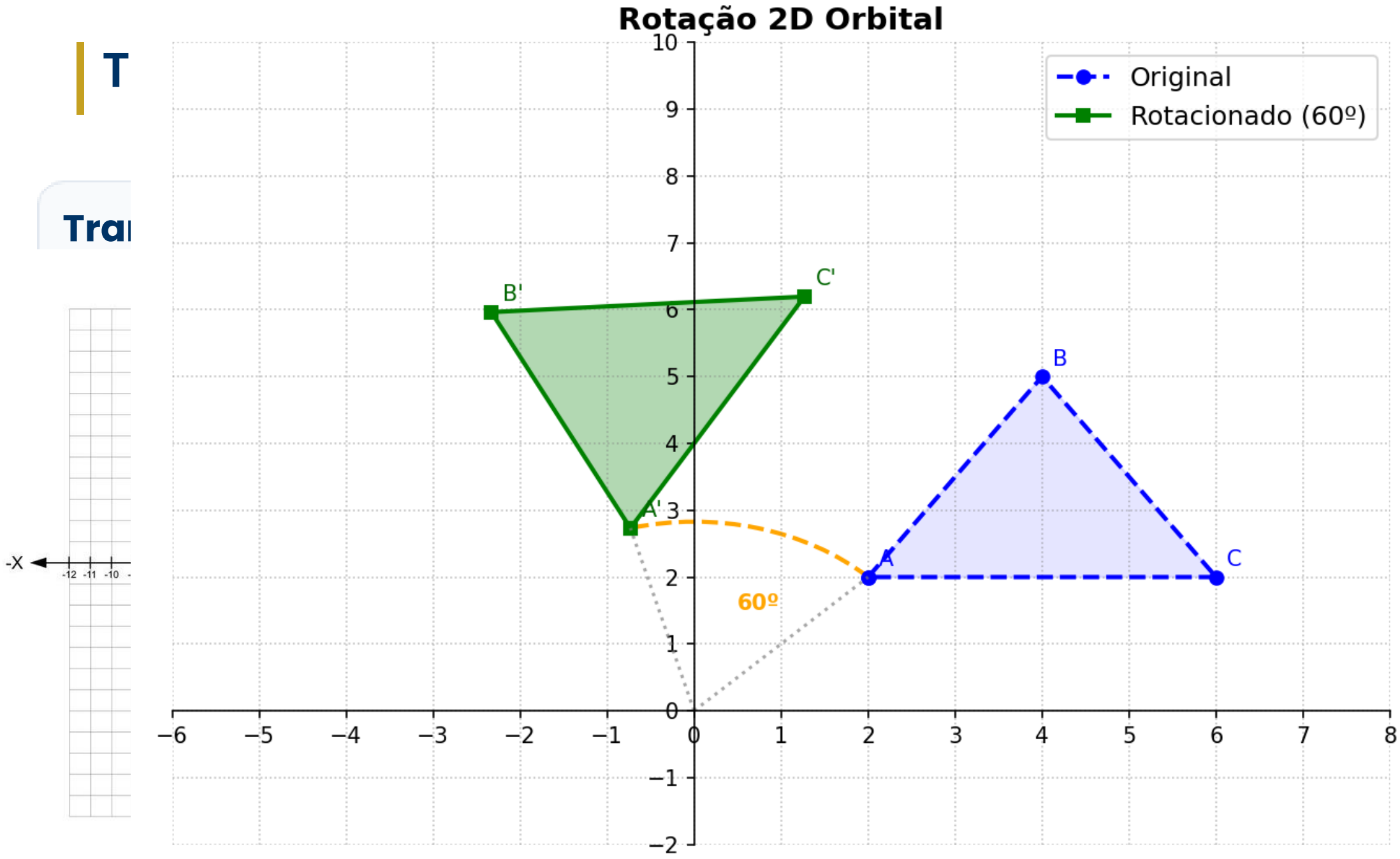
Transformação de Rotação: Na prática



transformacao_rotacao.py

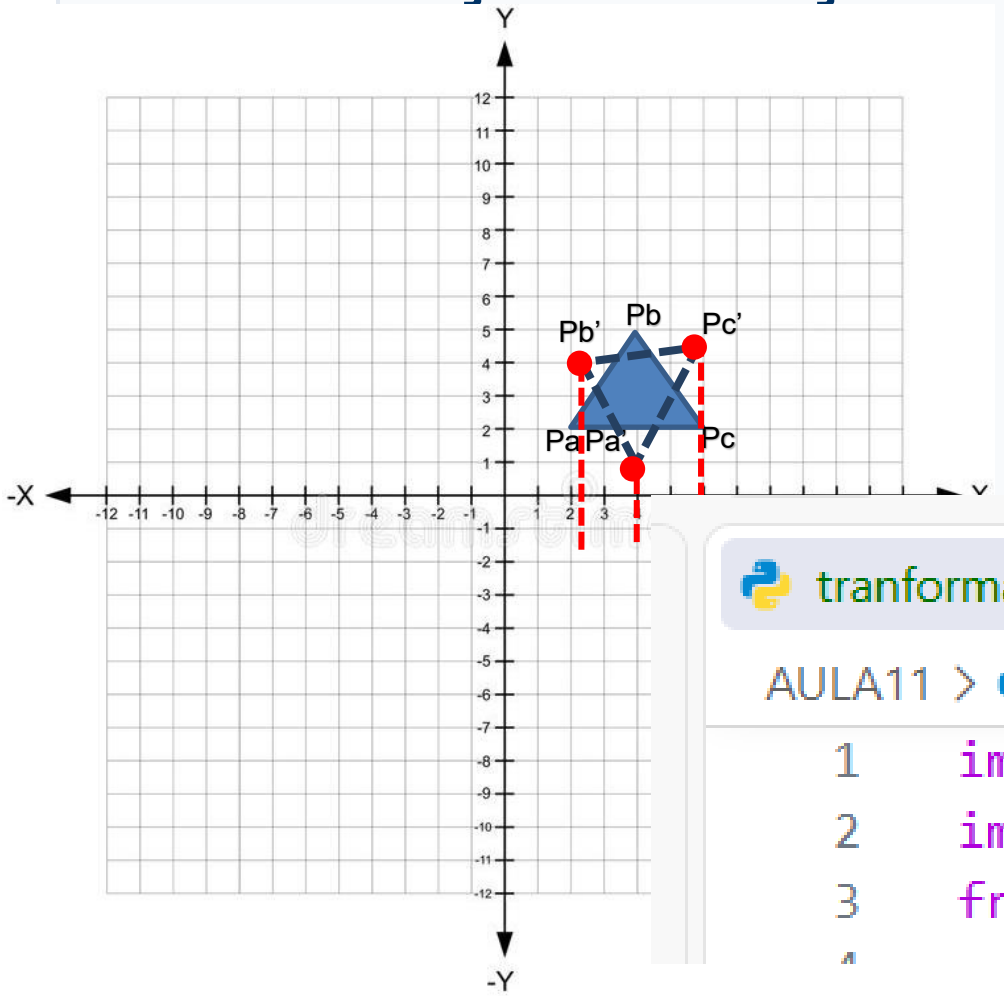
```
96
97      print("Renderizando o plano cartesiano...")
98      plt.show()
99
100 if __name__ == "__main__":
101     transformacao_rotacao()
```




Rotação 2D Orbital
Original
Rotacionado (60º)
B'
C'
A'
B
A
C
60º

Transformações Gráficas: Transformações Geométricas

Transformação de Rotação: Na prática



tranformacao_rotacao_origem.py

tranformacao_rotacao_origem.py U

AULA11 > tranformacao_rotacao_origem.py > rotacao_centro_gravidade

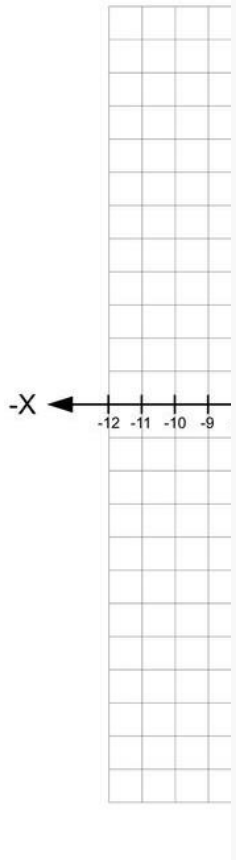
```
1   import numpy as np
2   import matplotlib.pyplot as plt
3   from matplotlib.patches import Arc
```

Transformações Gráficas: Transformações Geométricas

```
5   def rotacao_centro_gravidade():
6       print("=== Rotação no Centro de Gravidade ===\n")
7
8       # Definição da Geometria Original
9       # Vértices: A(2,2), B(4,5), C(6,2) e fechamento A(2,2)
10      matriz_vertices = np.array([
11          [2, 4, 6, 2], # Eixo X
12          [2, 5, 2, 2], # Eixo Y
13          [1, 1, 1, 1]  # Coordenada Homogênea (W)
14      ], dtype=np.float32)
15
16      # Cálculo do Centro de Gravidade (Baricentro)
17      cx = np.mean(matriz_vertices[0, 0:3])
18      cy = np.mean(matriz_vertices[1, 0:3])
19      print(f"Centro de Gravidade Calculado (Cx, Cy): ({cx:.2f}, {cy:.2f})\n")
```

Tr

Tran



tranformacao_rotacao_origem.py U

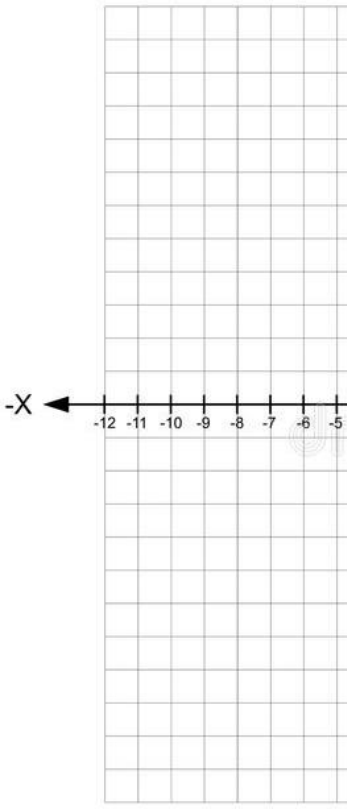
AULA11 > tranformacao_rotacao_origem.py > rotacao_centro_gravid

```
5   def rotacao_centro_gravidade():
20
21      # Construção das 3 Matrizes
22      angulo_graus = 60
23      theta = np.radians(angulo_graus)
24      cos_t, sin_t = np.cos(theta), np.sin(theta)
25
26      # Levar para a Origem (-Cx, -Cy)
27      M_T1 = np.array([
28          [1, 0, -cx],
29          [0, 1, -cy],
30          [0, 0,   1]
31      ])
```

Trans

eométricas

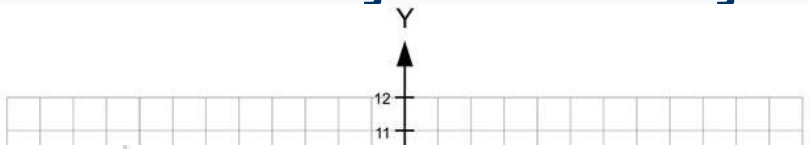
Transfor




```
33  # Rotacionar
34  M_R = np.array([
35      [cos_t, -sin_t, 0],
36      [sin_t,  cos_t, 0],
37      [    0,      0, 1]
38  ])
39
40  # Trazer de volta (Cx, Cy)
41  M_T2 = np.array([
42      [1, 0, cx],
43      [0, 1, cy],
44      [0, 0,  1]
45  ])
46
```

Transformações Gráficas: Transformações Geométricas

Transformação de Rotação: Na prática



tranformacao_rotacao_origem.py

```
47      # Matriz Mestre
48      # Dica: Ordem da Direita para a Esquerda! (M_T2 * M_R * M_T1)
49      M_Composta = M_T2 @ M_R @ M_T1
50
51      print("Matriz Composta Final (T2 * R * T1):")
52      print(np.round(M_Composta, 4), "\n")
53
54      # Multiplicando a matriz
55      matriz_transformada = M_Composta @ matriz_vertices
56
```

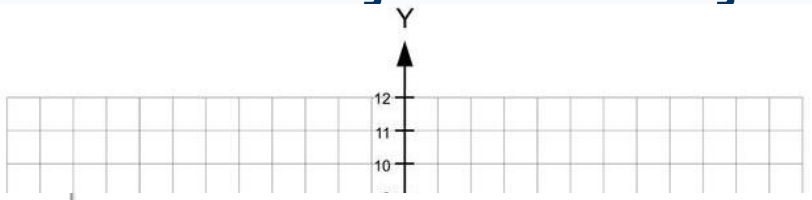
Transformações Gráficas: Transformações Geométricas

Transformação de Rotação: Na prática

```
57      # Renderização Visual no Matplotlib
58      fig, ax = plt.subplots(figsize=(10, 10))
59
60      x_orig, y_orig = matriz_vertices[0, :], matriz_vertices[1, :]
61      x_trans, y_trans = matriz_transformada[0, :], matriz_transformada[1, :]
62
63      # Plotando os triângulos
64      ax.plot(x_orig, y_orig, color='blue', linestyle='--',
65              linewidth=2, marker='o', label='Original')
66      ax.fill(x_orig, y_orig, color='blue', alpha=0.1)
67
68      ax.plot(x_trans, y_trans, color='green', linestyle='-',
69              linewidth=2, marker='s', label=f'Rotacionado ({angulo_graus}º) no Eixo')
70      ax.fill(x_trans, y_trans, color='green', alpha=0.3)
71
```

Transformações Gráficas: Transformações Geométricas

Transformação de Rotação: Na prática



tranformacao_rotacao_origem.py

```
    # Plotando e Anotando o Centro de Gravidade
    ax.plot(cx, cy, color='black', marker='X', markersize=10, label='Centro de Gravidade')
    ax.annotate(f" CG ({cx:.1f}, {cy:.1f})", (cx, cy),
                textcoords="offset points", xytext=(5,-15), color='black', fontweight='bold')

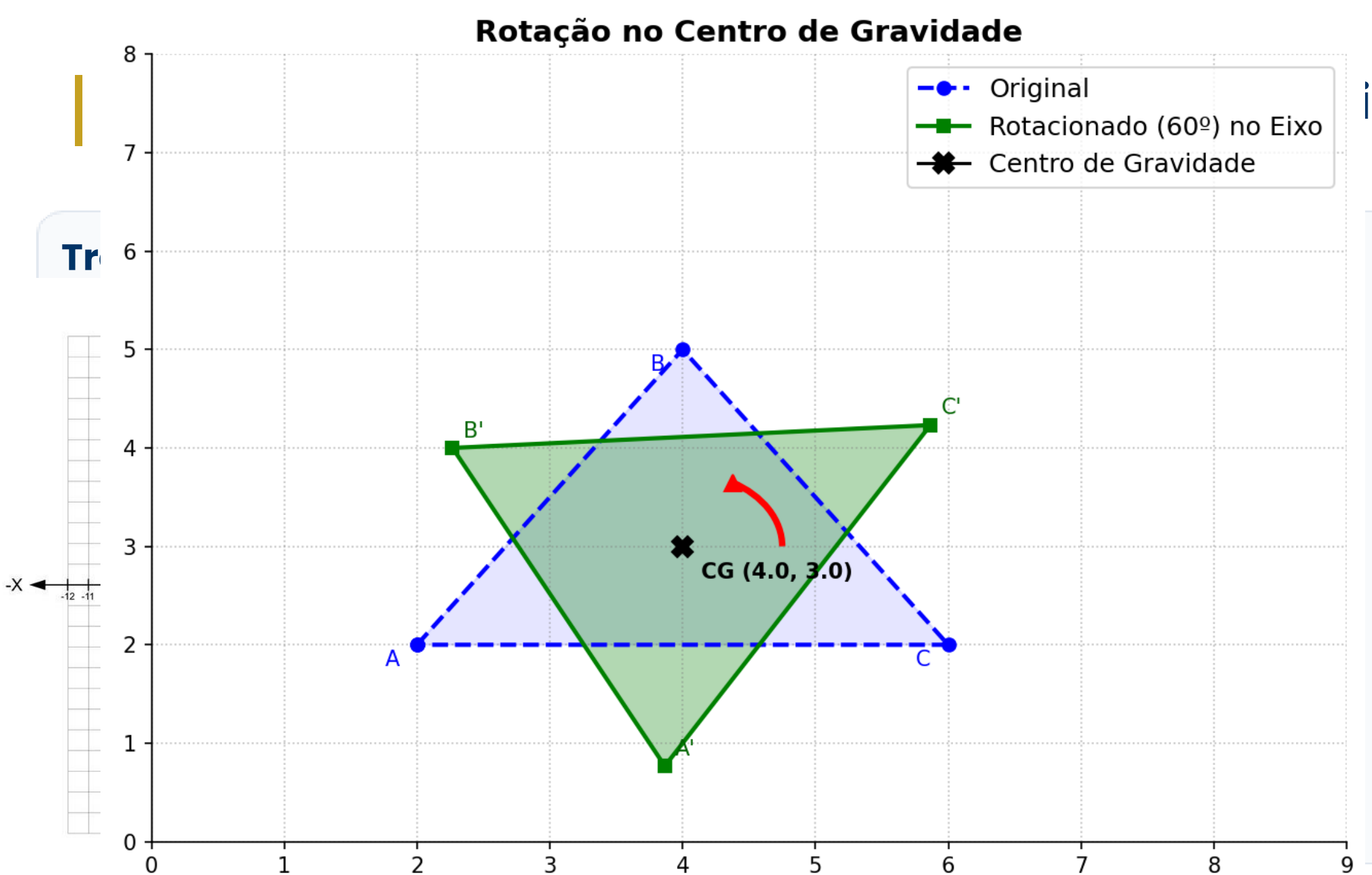
    # Legendas
    rotulos = ['A', 'B', 'C']
    for i in range(3):
        ax.annotate(f"{rotulos[i]}", (x_orig[i], y_orig[i]),
                    textcoords="offset points", xytext=(-15,-10), color='blue')
        ax.annotate(f"{rotulos[i]}'", (x_trans[i], y_trans[i]),
                    textcoords="offset points", xytext=(5,5), color='darkgreen')
```

Transformações Gráficas: Transformações Geométricas

```
85      # Desenhando um arco demonstrando o giro NO PRÓPRIO EIXO
86      arco = Arc((cx,cy), 1.5, 1.5, angle=0, theta1=0,
87                 theta2=angulo_graus, color='red', linewidth=3, linestyle='-')
88      ax.add_patch(arco)
89      # Ponta da seta do arco (aproximada)
90      ax.plot([cx + 0.75*np.cos(theta)],
91              [cy + 0.75*np.sin(theta)], marker='^', color='red', markersize=8)
92
93      # Configuração do Plano Cartesiano
94      ax.set_title("Rotação no Centro de Gravidade", fontsize=14, fontweight='bold')
95      ax.set_xlim(0, 9)
96      ax.set_ylim(0, 8)
97
98      ax.spines['left'].set_position('zero')
99      ax.spines['bottom'].set_position('zero')
100     ax.spines['right'].set_color('none')
101     ax.spines['top'].set_color('none')
```

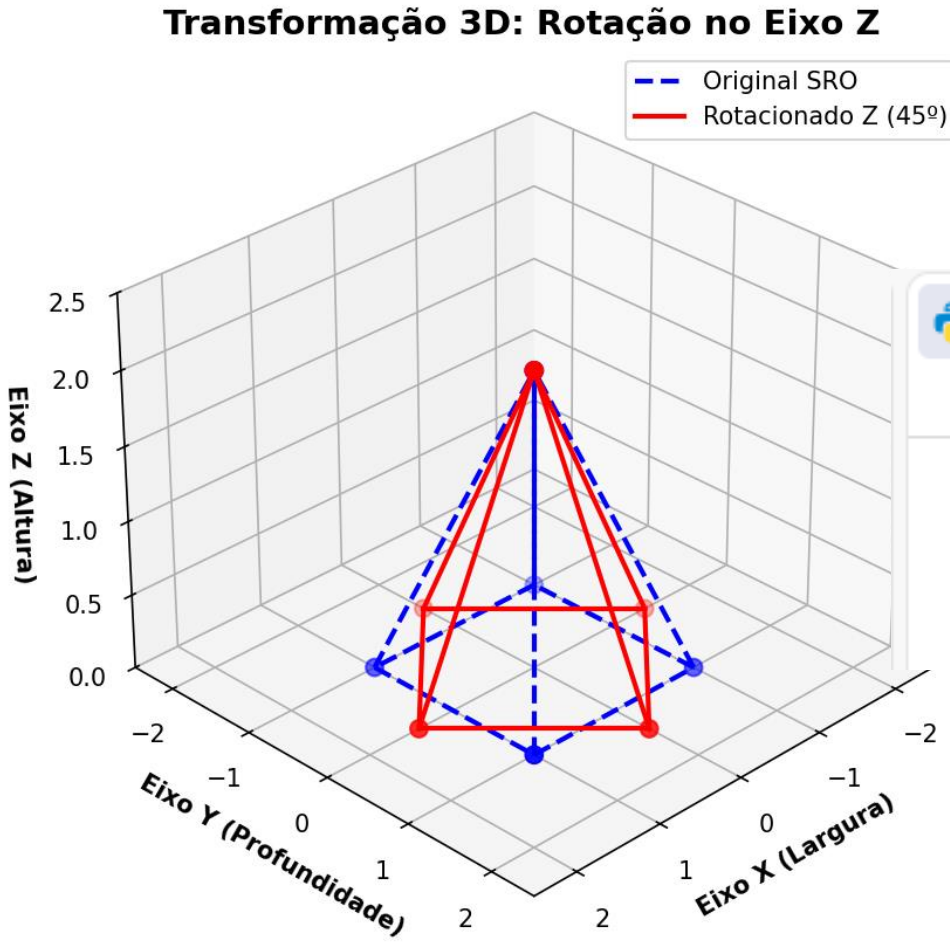
Transformações Gráficas: Transformações Geométricas

```
102
103         ax.set_xticks(np.arange(0, 10, 1))
104         ax.set_yticks(np.arange(0, 9, 1))
105         ax.grid(True, linestyle=':', color='gray', alpha=0.5)
106
107         ax.legend(loc='upper right', fontsize=12)
108
109         print("Renderizando o plano cartesiano...")
110         plt.show()
111
112     if __name__ == "__main__":
113         rotacao_centro_gravidade()
```


Rotação no Centro de Gravidade
Original
Rotacionado (60º) no Eixo
Centro de Gravidade
A
B
C
A'
B'
C'
CG (4.0, 3.0)
0
1
2
3
4
5
6
7
8
9

Transformações Gráficas: Transformações Geométricas

Transformação de Rotação: Na prática



transformacao_rotacao_3d_exemplo01.py

transformacao_rotacao_3d_exemplo01.py U

AULA11 > transformacao_rotacao_3d_exemplo01.py > plotar_rotacao_3d_eixo_z

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3  from mpl_toolkits.mplot3d import Axes3D
4
```

Transformações Gráficas: Transformações Geométricas

Transformação de Rotação: Na prática

```
5   def plotar_rotacao_3d_eixo_z():
6       print("=== Rotação 3D no Eixo Z ===\n")
7
8       # Pirâmide de base quadrada centrada na origem
9       # Vértices: A(Base Esq Frontal), B(Base Dir Frontal), C(Base Dir Tras),
10      # D(Base Esq Tras), E(Topo/Ápice)
11      matriz_vertices = np.array([
12          [-1,  1,  1, -1,  0],  # X
13          [-1, -1,  1,  1,  0],  # Y
14          [ 0,  0,  0,  0,  2],  # Z (A base está no chão Z=0, o topo está em Z=2)
15          [ 1,  1,  1,  1,  1]   # W (Coordenada Homogênea)
16      ], dtype=np.float32)
17
```

```python
# Matriz 4x4 de Rotação em Z
angulo_graus = 45
theta = np.radians(angulo_graus)
cos_t, sin_t = np.cos(theta), np.sin(theta)

matriz_rotacao_z = np.array([
    [ cos_t, -sin_t, 0, 0],
    [ sin_t,  cos_t, 0, 0],
    [     0,      0, 1, 0],
    [     0,      0, 0, 1]
])

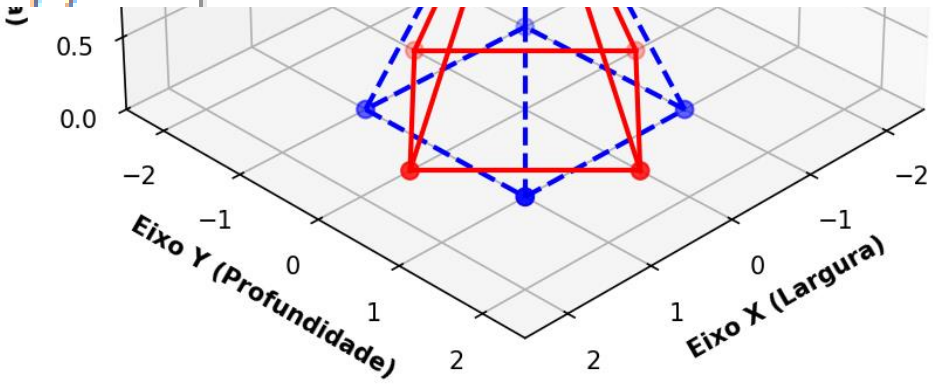
print(f"Matriz de Rotação 4x4 (Eixo Z, {angulo_graus}º):")
print(np.round(matriz_rotacao_z, 4), "\n")
```

Eixo Z (Altura)

Transformações Gráficas: Transformações Geométricas

Transformação de Rotação: Na prática

```
33      # Multiplicação
34      matriz_transformada = matriz_rotacao_z @ matriz_vertices
35
36      # Renderização 3D
37      fig = plt.figure(figsize=(12, 8))
38      ax = fig.add_subplot(111, projection='3d')
39
```



```python
# Função auxiliar para desenhar as arestas do objeto
def desenhar_wireframe(ax, matriz, cor, label, estilo):
    x, y, z = matriz[0, :], matriz[1, :], matriz[2, :]

    # Lista de conexões (índices dos vértices para desenhar as linhas)
    # Base: 0->1->2->3->0, Arestas laterais: 0->4, 1->4, 2->4, 3->4
    arestas = [
        (0, 1), (1, 2), (2, 3), (3, 0), # Base
        (0, 4), (1, 4), (2, 4), (3, 4)  # Laterais até o topo
    ]

    # Plota os vértices (Pontos)
    ax.scatter(x, y, z, color=cor, s=50)

    # Plota as arestas (Linhas)
    for i, (inicio, fim) in enumerate(arestas):
        lbl = label if i == 0 else "" # Evita repetir label na legenda
        ax.plot([x[inicio], x[fim]],
                [y[inicio], y[fim]],
                [z[inicio], z[fim]],
                color=cor, linestyle=estilo, linewidth=2, label=lbl)
```

Transformações Gráficas: Transformações Geométricas

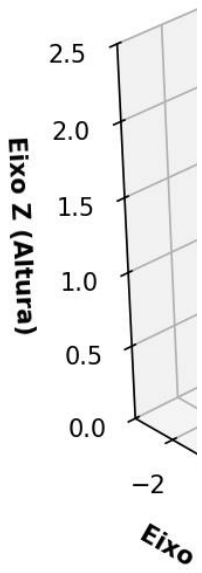
```
62      # Desenhando a Pirâmide Original (Azul, tracejada)
63      desenhar_wireframe(ax, matriz_vertices, 'blue', 'Original SRO', '--')
64
65      # Desenhando a Pirâmide Rotacionada (Vermelha, sólida)
66      desenhar_wireframe(ax, matriz_transformada, 'red',
67                         f'Rotacionado Z ({angulo_graus}º)', '-')
68
69      # Configuração do Espaço Cartesiano 3D
70      ax.set_title("Transformação 3D: Rotação no Eixo Z",
71                   fontsize=14, fontweight='bold')
72      ax.set_xlabel("Eixo X (Largura)", fontweight='bold')
73      ax.set_ylabel("Eixo Y (Profundidade)", fontweight='bold')
74      ax.set_zlabel("Eixo Z (Altura)", fontweight='bold')
75
```

Tr

eométricas

Tran

Tr



01.py

```python
    # Travando a escala dos eixos
    limite = 2.5
    ax.set_xlim([-limite, limite])
    ax.set_ylim([-limite, limite])
    ax.set_zlim([0, limite])

    # Definindo a câmera inicial
    ax.view_init(elev=30, azim=45)

    ax.legend()
    plt.show()

if __name__ == "__main__":
    plotar_rotacao_3d_eixo_z()
```

Dúvidas e Discussão

Este é o momento para aprofundarmos nos conceitos.

raulivan@cefetmg.br

SIGAA

Vamos praticar

Desafio do Engenheiro de Satélites Espaciais

Como um engenheiro de software de uma agência espacial, precisa modelar as peças do satélite:

- um corpo
- dois painéis solares
- e uma antena

Vamos praticar

Desafio do Engenheiro de Satélites Espaciais

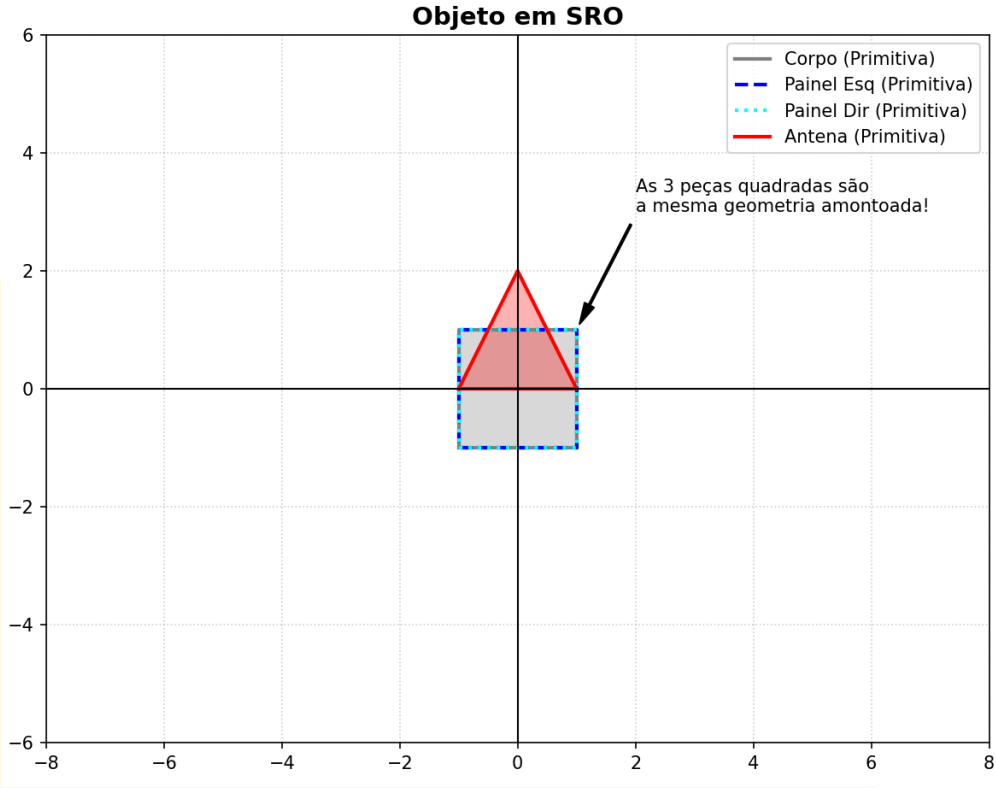
Porém, todas as peças foram modeladas na origem (0,0), no Sistema de Referência do Objeto (SRO).
A sua missão matemática é compor matrizes de Translação, Rotação e Escala (TRS) para montar o satélite no espaço (SRU).

Vamos praticar

Desafio do Engenheiro de Satélites Espaciais

Objetos a serem usados:

- Corpo: p1(-1,-1), p2(1,-1),p3(1,1),p4(-1,1)
- Painéis solares: p1(-1,-1), p2(1,-1),p3(1,1),p4(-1,1)
- Antena: p1(-1,0),p2(1,0),p3(0,2)

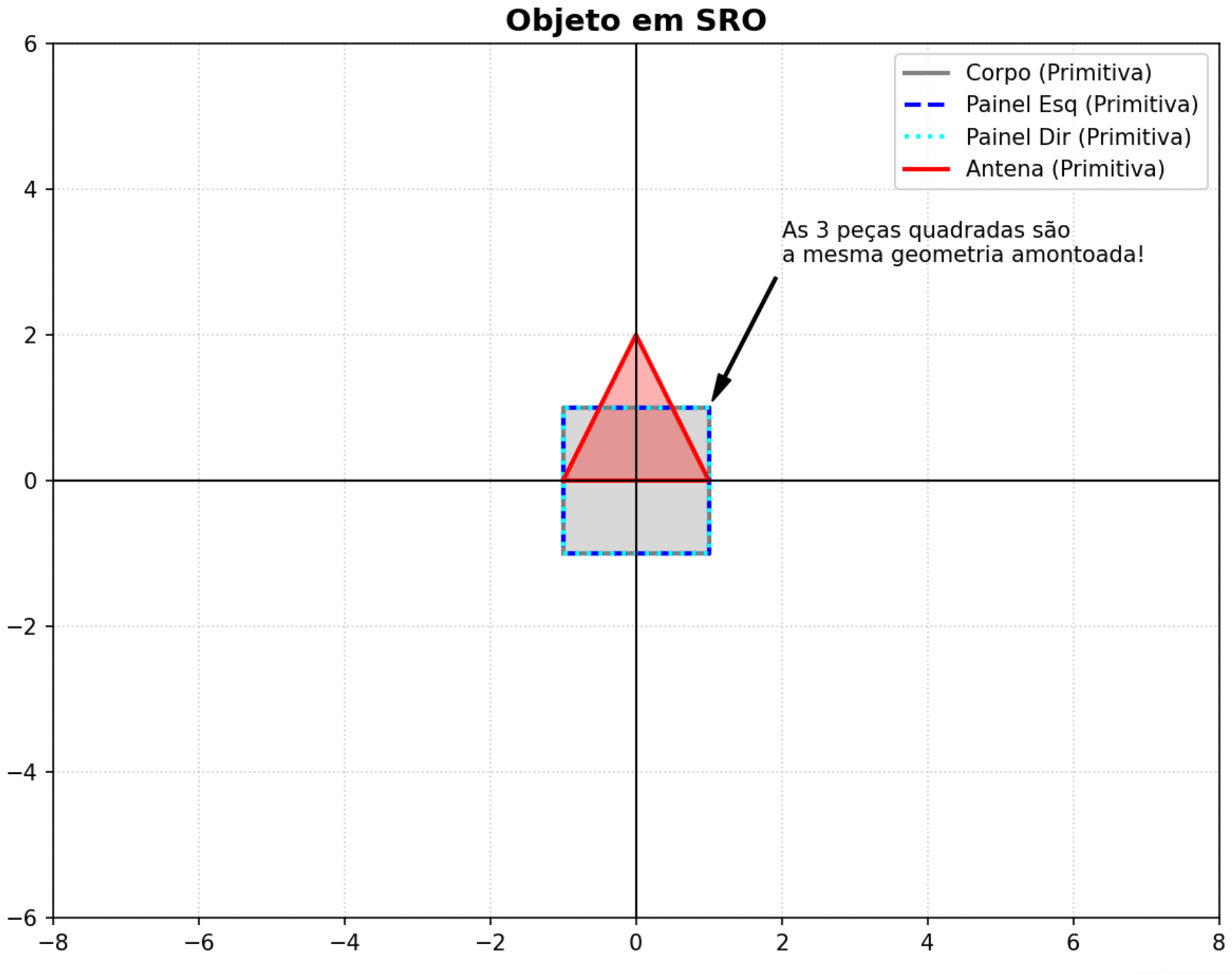


Vam

Desc

Objet

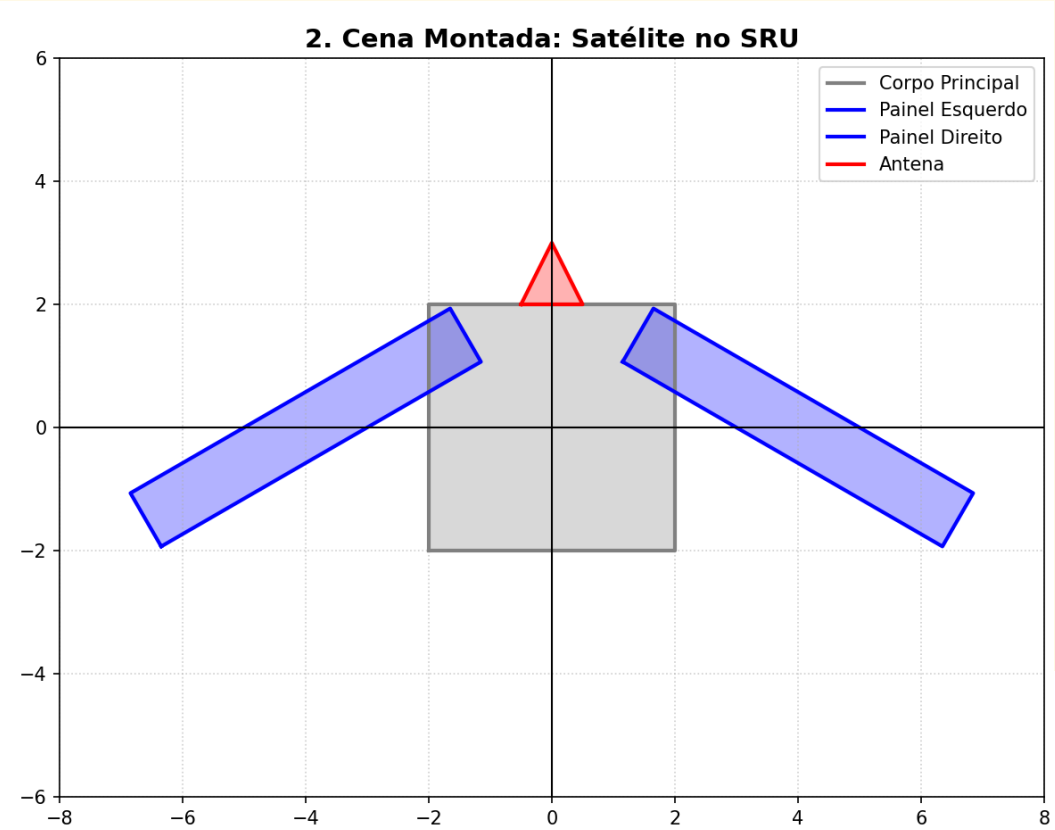
- Co
- Pa
- An



Vamos praticar

Desafio do Engenheiro de Satélites Espaciais

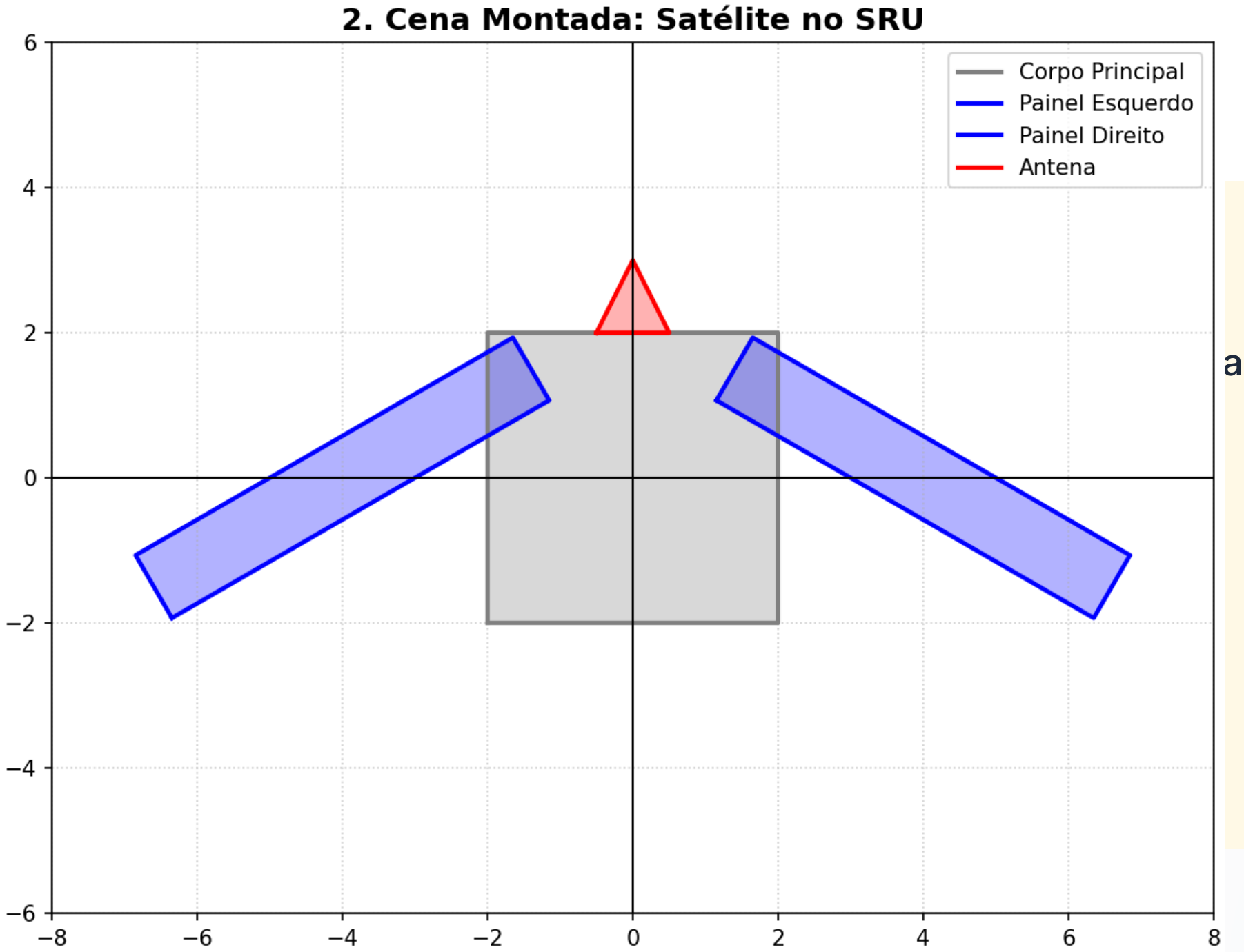
Você deve aplicar as transformações necessárias pra gerar o seguinte resultado:



Vamo

Desa

Você ado:



Próxima Aula

Transformações Gráficas

